

Non-volatile Storage & Streamline Data Exchanges (2)

Hung-Wei Tseng

QLC = More Density Per NAND Cell



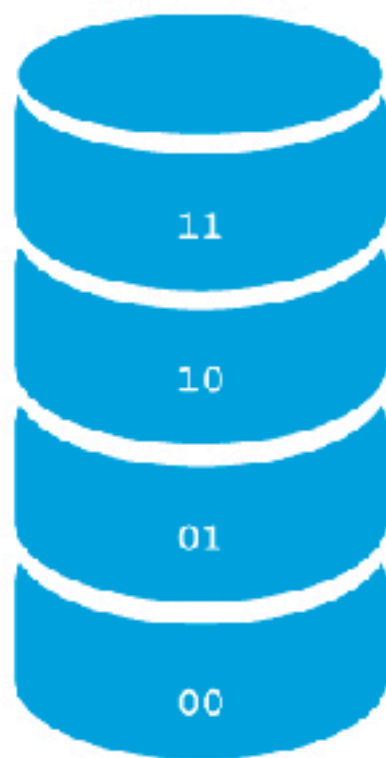
SLC



1 Bit Per Cell

First SSD NAND technology

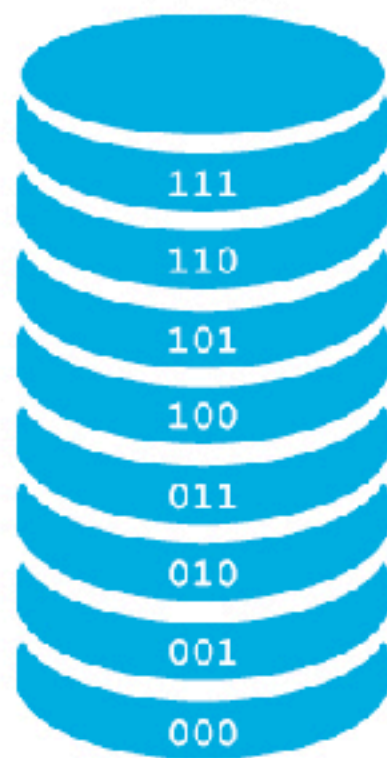
MLC



2 Bits Per Cell

100% increase

TLC



3 Bits Per Cell

50% increase

QLC



4 Bits Per Cell

33% increase



100K P/E Cycles
(at technology introduction)

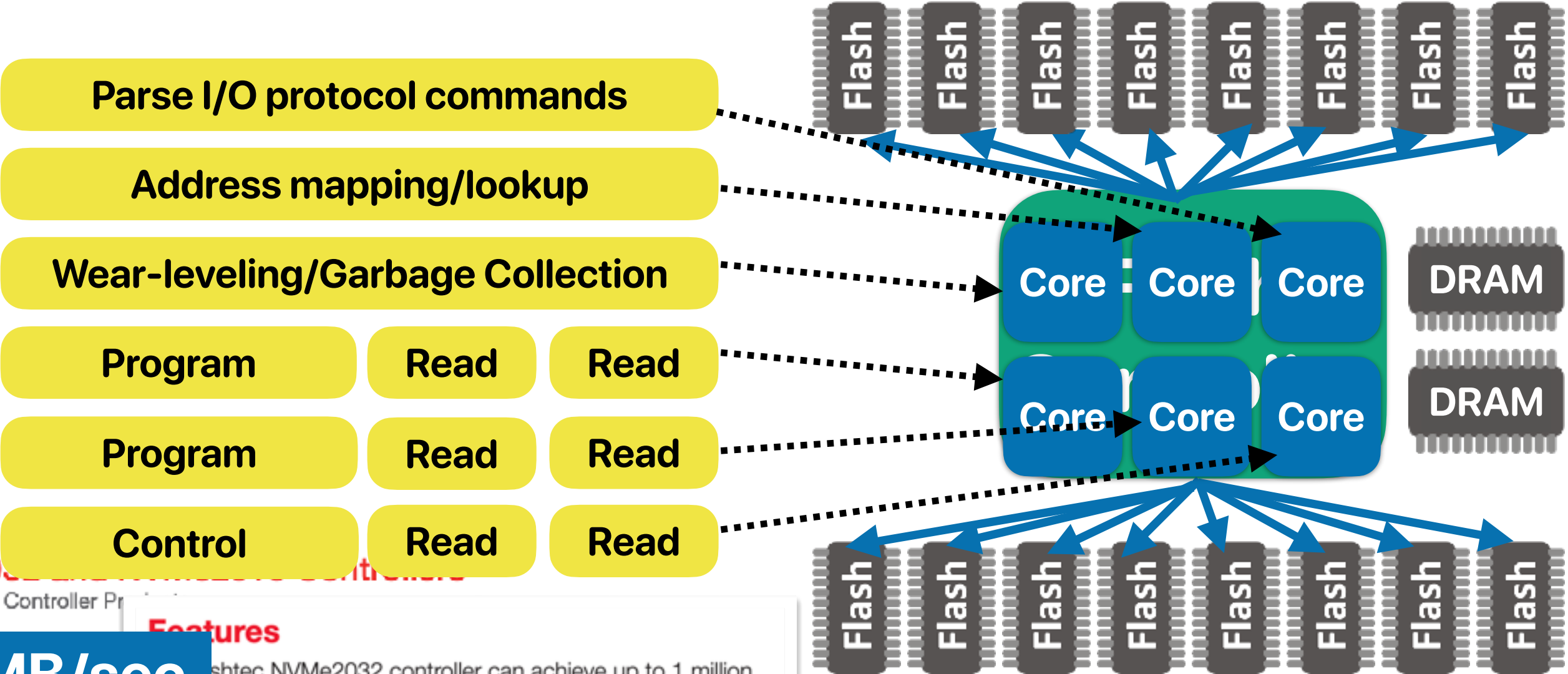
10K P/E Cycles

3K P/E Cycles

1K P/E Cycles

Fewer writes per cell

Recap: Multi-channel to optimize bandwidth



Each ~267MB/sec
16 channels ==
4.2GB/sec

Flashtec™ NVMe2032 32- and 16-Channel PCIe Flash Controller Pr

Features

Flashtec NVMe2032 controller can achieve up to 1 million random read IOPS on 4 KB operations

to 20 TB Flash capacity using 256 GB Flash

C, MLC, Enterprise MLC, and TLC Flash with to

FI interface

le Gen 3 x8 or dual independent PCIe Gen 3 x4 (active,

ative/standby) host interface

- 16 and 32 independent Flash channels, each suppo up to 8 CE



- Organization
 - Page size x8: 18,592 bytes (16,384 + 2208 bytes)
 - Block size: 2304 pages, (36,864K + 4968K bytes)
 - Plane size: 4 planes x 504 blocks
 - Device size: 512Gb: 2016 blocks; 1Tb: 4032 blocks; 2Tb: 8064 blocks; 4Tb: 16,128 blocks; 8Tb: 32,256 blocks

TLC 512Gb-8Tb NAND
- Array Characteristics

is suspended or ongoing					
READ PAGE operation time without/with V _{pp}	^t R	57/41	60	μs	2, 12
SNAP READ operation time	^t RSNAP	27	37	μs	
Cache read busy time	^t RCBSY	11	60	μs	2, 8, 12

Recap: Right protocol for SSDs

- Interrupt or polling?
- DMA or PIO?
- What other characteristics that conventional HDD cannot support?

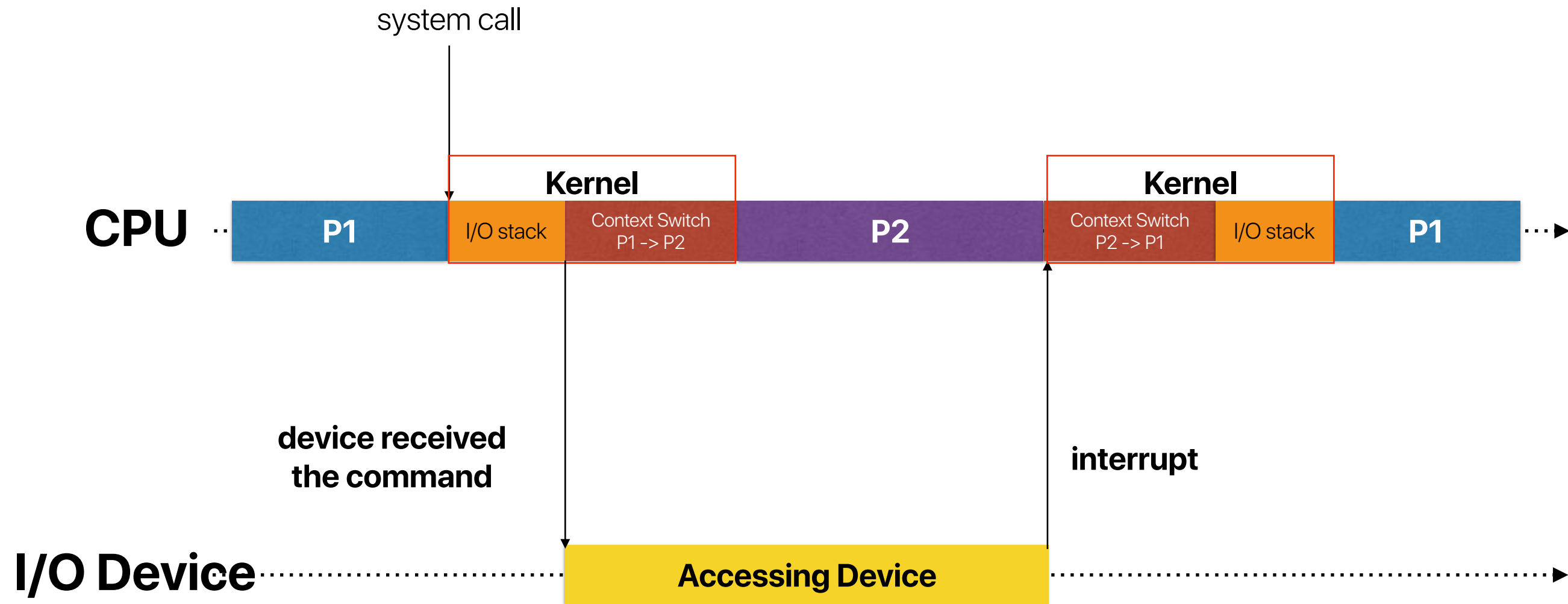
Multiple, parallel access at the same time!

What happened during a context switch & how long would that take?

Recap: What happens during context switch

- Load architectural states from process control block (somewhere in the main memory, potentially a cache miss, TLB miss) — takes several microseconds if everything is in the physical memory — **60 ns at least**
- Set processor registers according to the loaded architectural states
 - **Load the root page table of the restored/new process — 60 ns at least**
 - **Set the CR3 (page table base register in x86) register to identify the root page table node in the hierarchical page table**
 - Set the RIP (program counter in x86) to the previous execution
- **Restore virtual memory address**
 - **TLB flush**
 - Invalidate all entries in the TLB
 - Most TLBs are not tagged, so you've to do this
- You DO NOT have to load every page content back from disk — remember that we have demand paging!

To switch or not to switch that's the question.

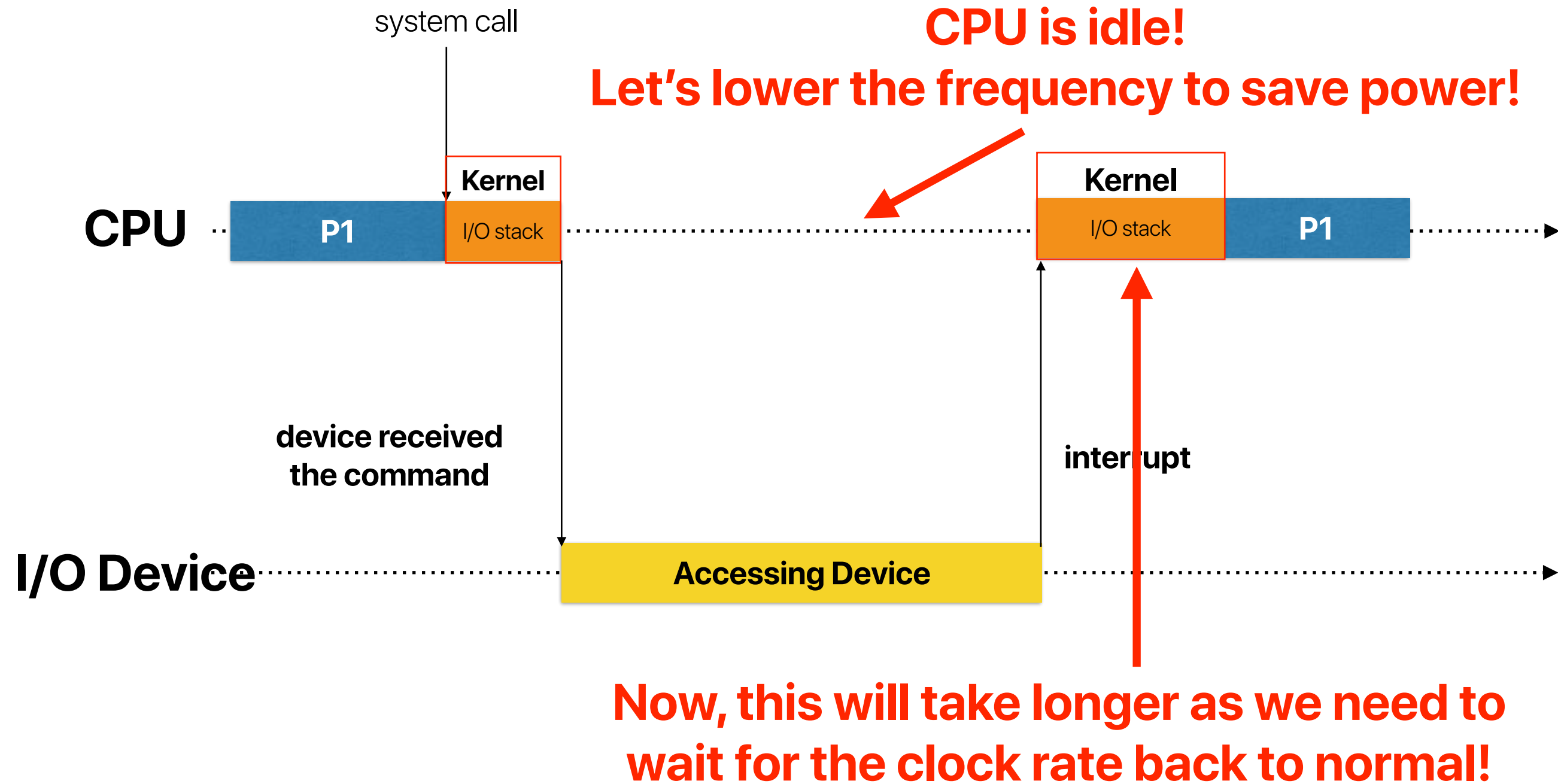


If $T_{\text{Context switch P1} \rightarrow \text{P2}} + T_{\text{Context switch P2} \rightarrow \text{P1}} < T_{\text{Accessing peripherals}}$
makes sense to context switch

But context switch overhead is not the only thing

- Cache warm up cost when you switch back
 - TLB warm up cost
- each memory access is potentially a page fault + TLB miss + cache miss

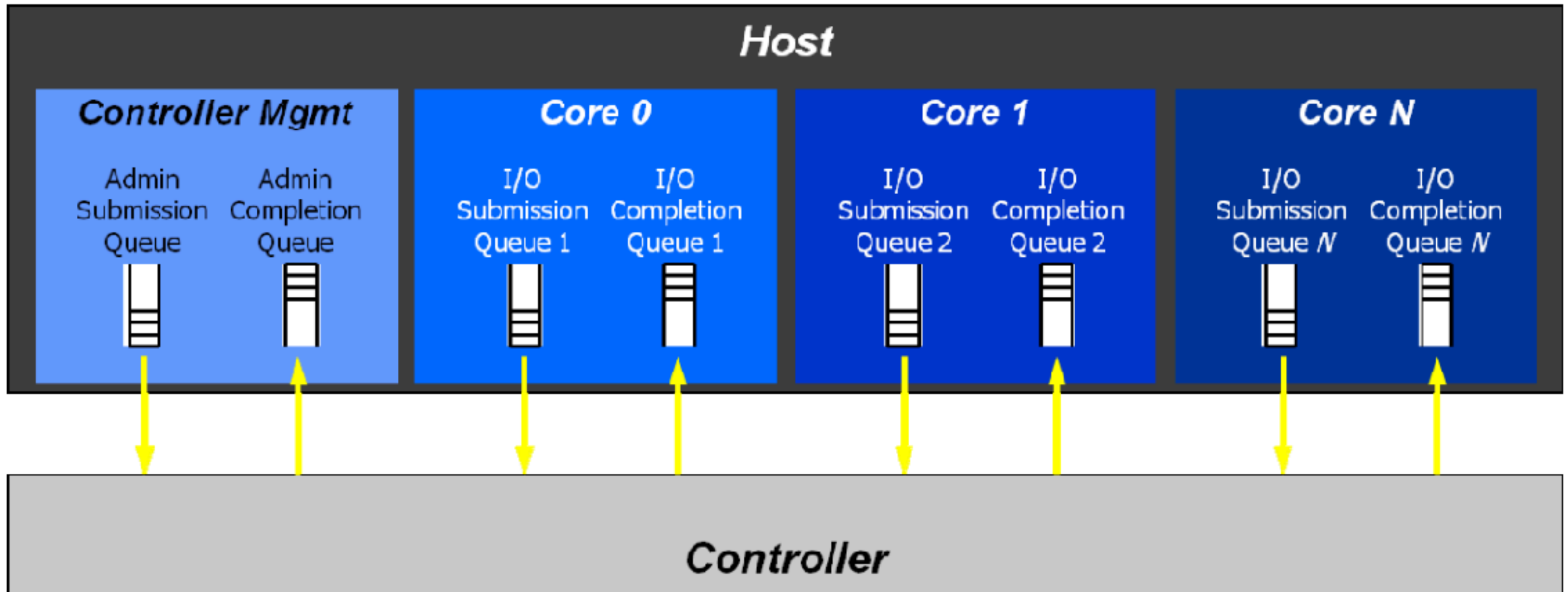
What if we don't switch?



What can you do to take advantage of multiple access channels in SSDs?

NVMe Model

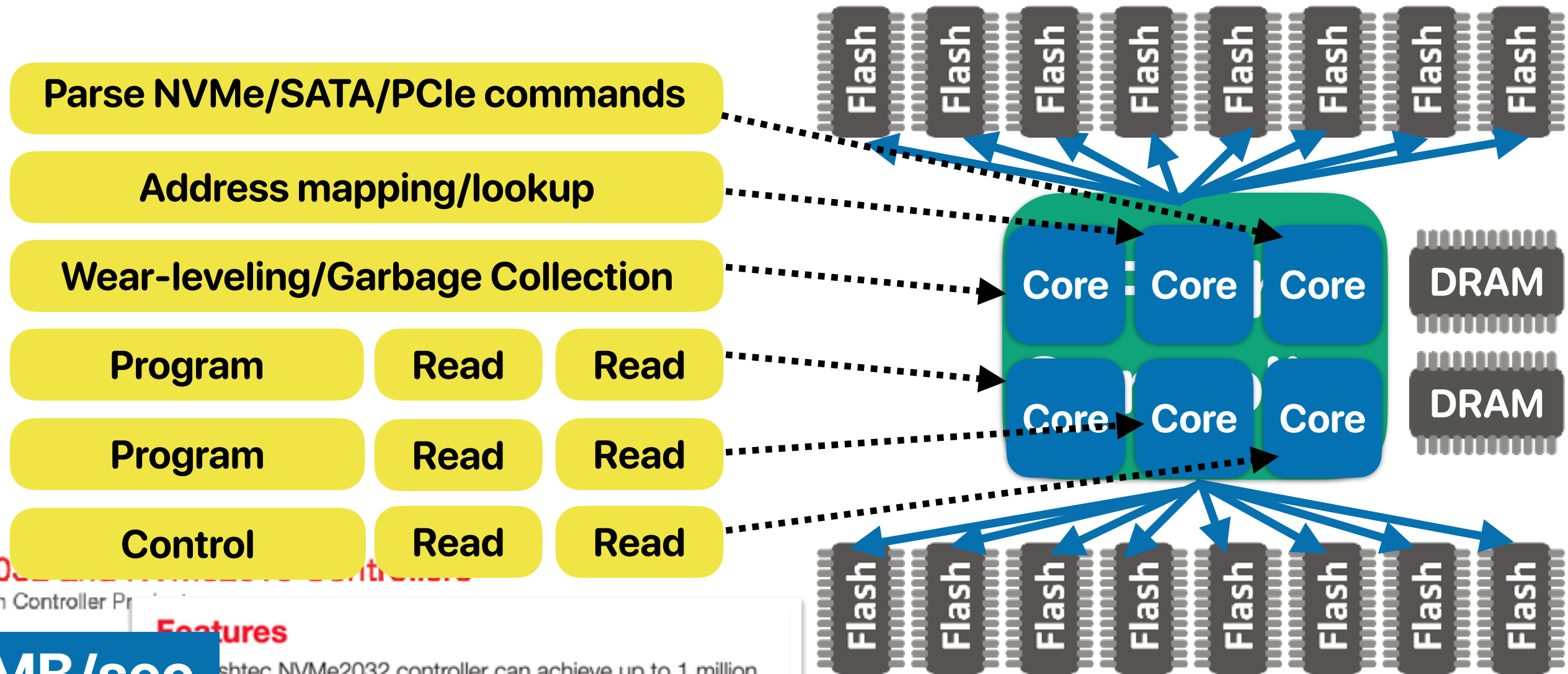
Figure 3: Queue Pair Example, 1:1 Mapping



https://nvmexpress.org/wp-content/uploads/2013/04/NVM_whitepaper.pdf

Why does MQ make sense?

Multi-channel to optimize bandwidth



Each ~267MB/sec
16 channels ==
4.2GB/sec

Flashtec™ NVMe2032
32- and 16-Channel PCIe Flash Controller Pr

Features

Flashtec NVMe2032 controller can achieve up to 1 million random read IOPS on 4 KB operations
to 20 TB Flash capacity using 256 GB Flash
C, MLC, Enterprise MLC, and TLC Flash with to
FI interface
le Gen 3 x8 or dual independent PCIe Gen 3 x4 (active,
ative/standby) host interface

- 16 and 32 independent Flash channels, each support up to 8 CE



Organization

- Page size x8: 18,592 bytes (16,384 + 2208 bytes)
- Block size: 2304 pages, (36,864K + 4968K bytes)
- Plane size: 4 planes x 504 blocks
- Device size: 512Gb: 2016 blocks; 1Tb: 4032 blocks; 2Tb: 8064 blocks; 4Tb: 16,128 blocks; 8Tb: 32,256 blocks

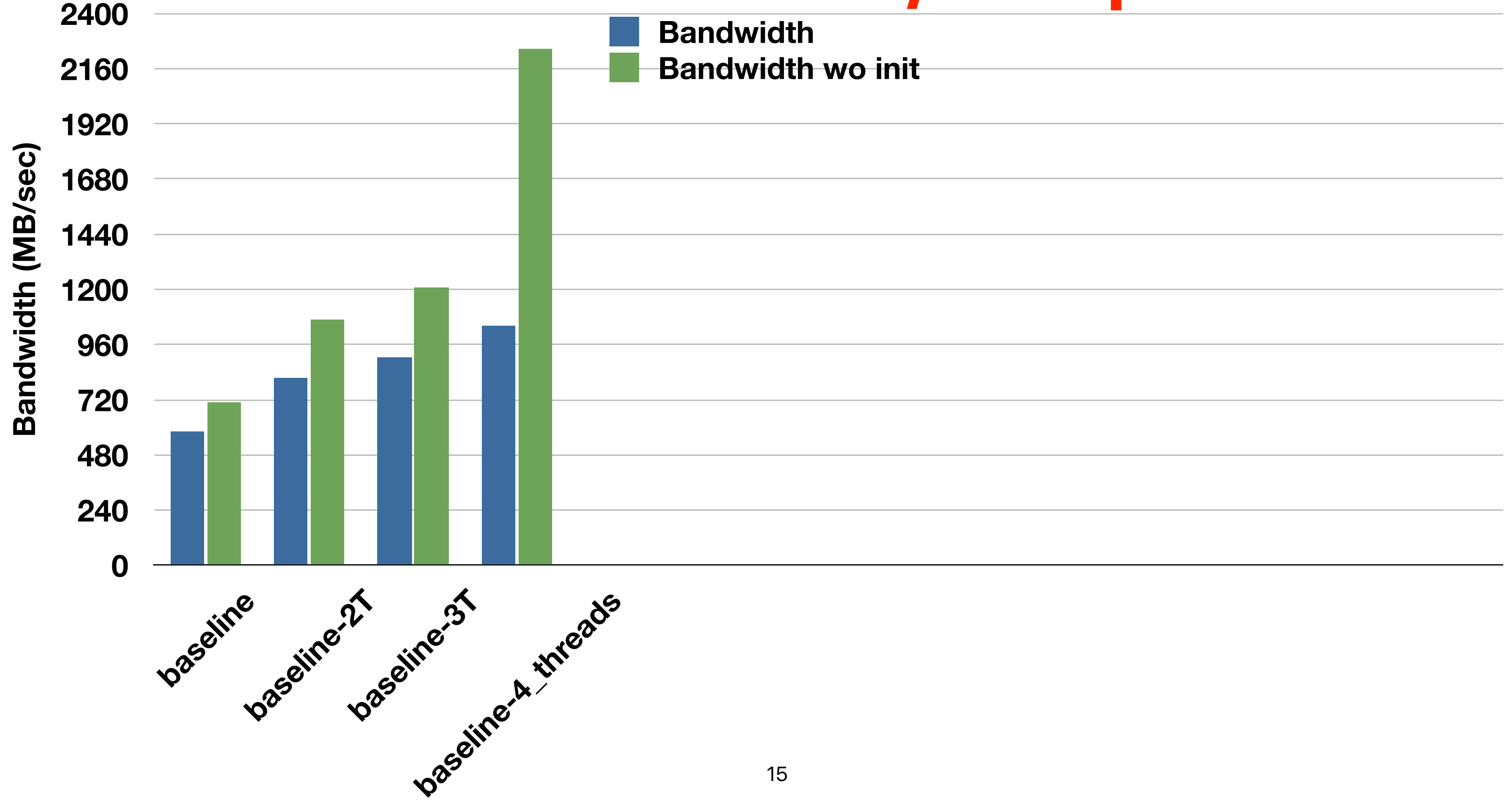
TLC 512Gb-8Tb NAND Array Characteristics

is suspended or ongoing					
READ PAGE operation time without/with V _{pp}	^t R	57/41	60	μs	2, 12
SNAP READ operation time	^t RSNAP	27	37	μs	
Cache read busy time	^t RCBSY	11	60	μs	2, 8, 12

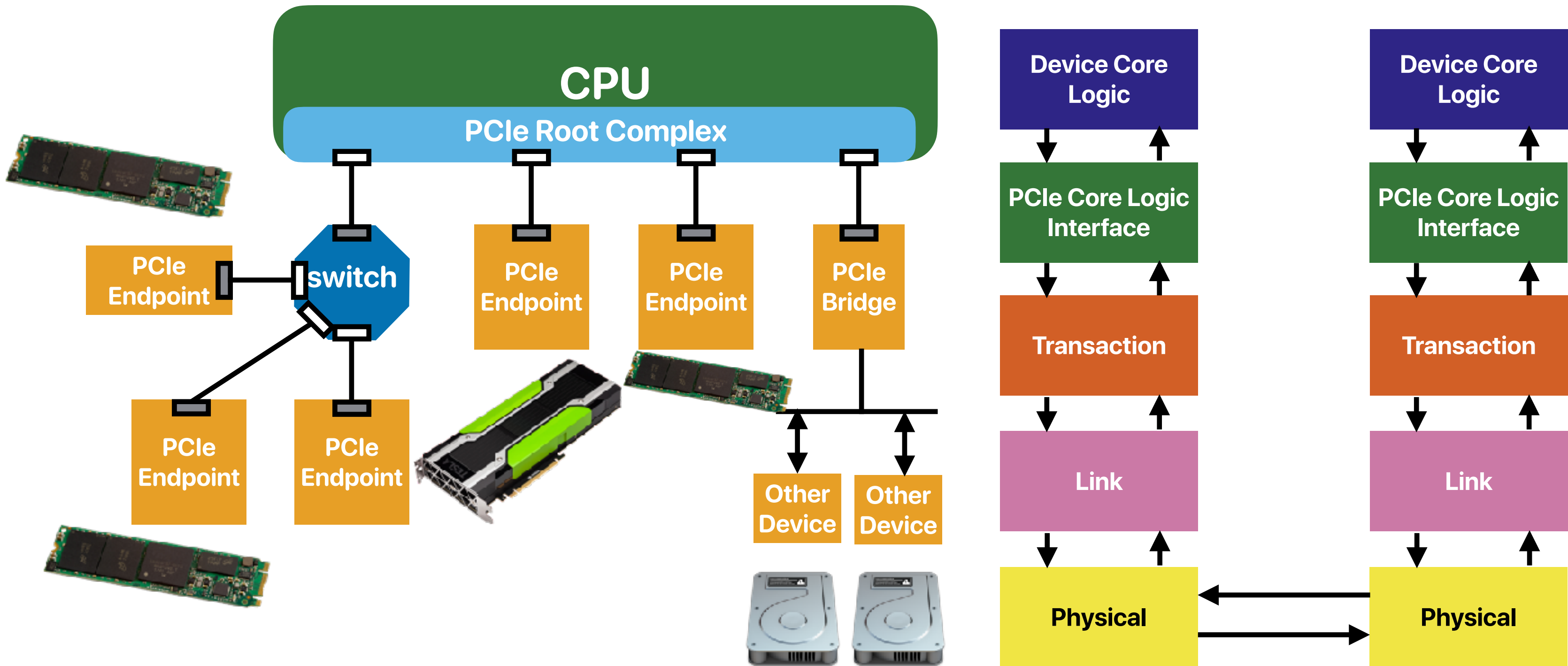
Why does MQ makes sense?

- Flash SSDs have multiple channels inside!
 - You need multiple requests to fill the requests
- The same design cannot work on HDDs — HDDs have only one head

Multi-threaded I/O helps

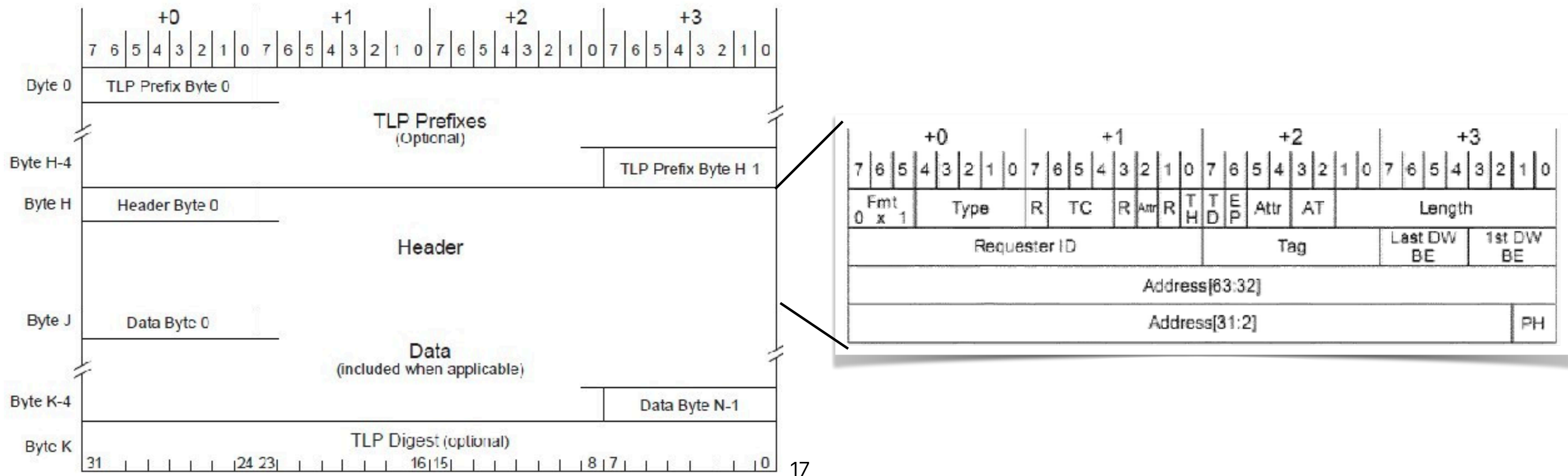


PCIe "Interconnect"



PCIe interconnect

- Very similar to computer networks
- Use "memory addresses" as the identifier for routing
- Peer-to-peer communication is possible



NVMe

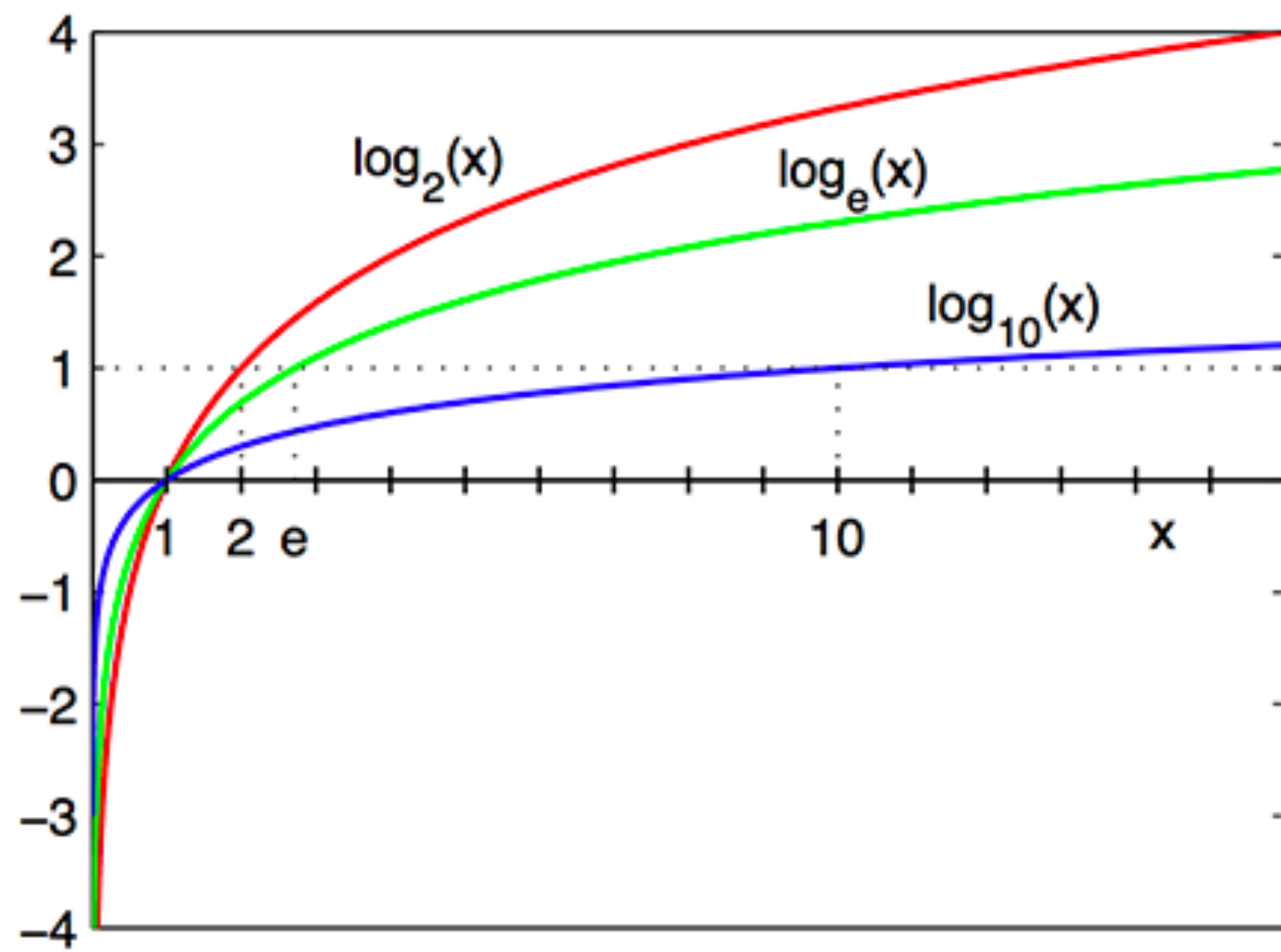
- The standard of PCIe SSD devices now
 - Provides multiple command queues to better support multithreading hardware
 - Allows more parallelism inside the SSD
- The “payload” of a PCIe packet

0	8	16	32	48	63
OPCODE	FLAGS	command id	Namespace ID		
reserved					
metadata					
PRP1					
PRP2					
Start LBA					
length		control	Dataset management		
Reference tag			App tag		App mask

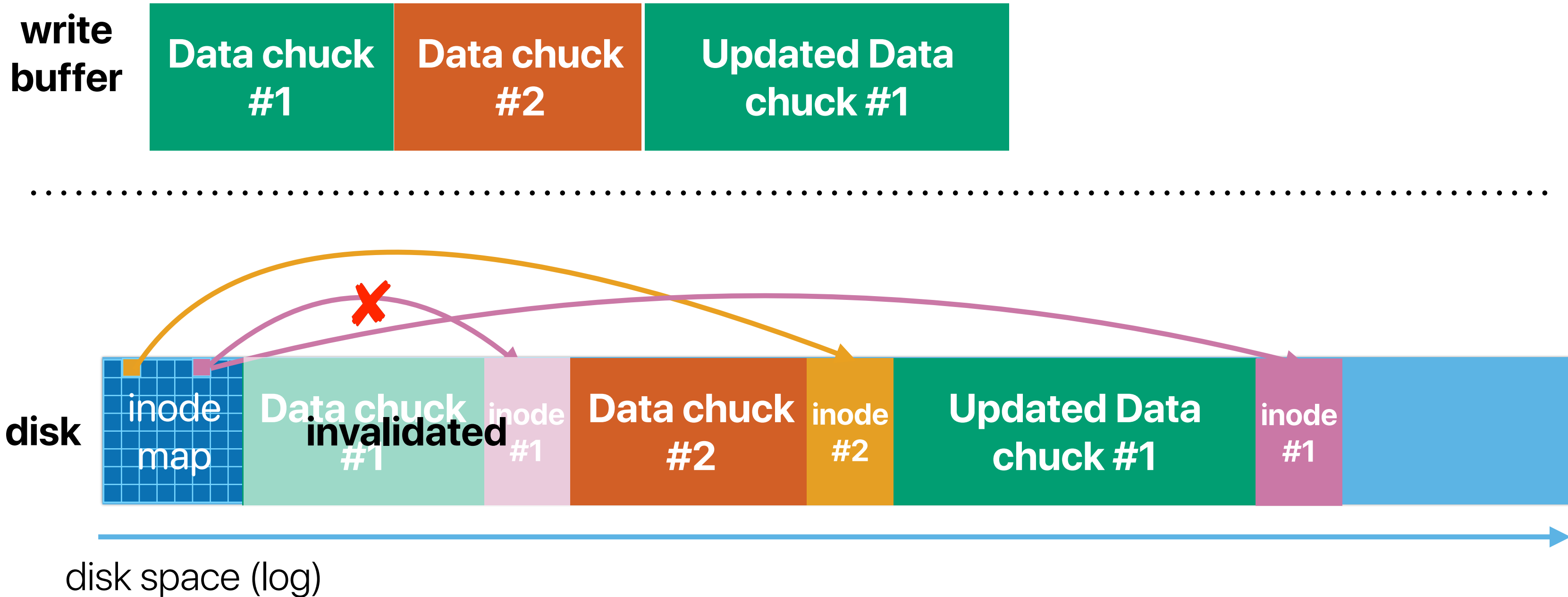
Let's walk through an NVMe driver

Don't stack your log on my log

**Jingpei Yang, Ned Plasson, Greg Gillis, Nisha Talagala, and
Swaminathan Sundararaman
SanDisk Corporation**



LFS in motion

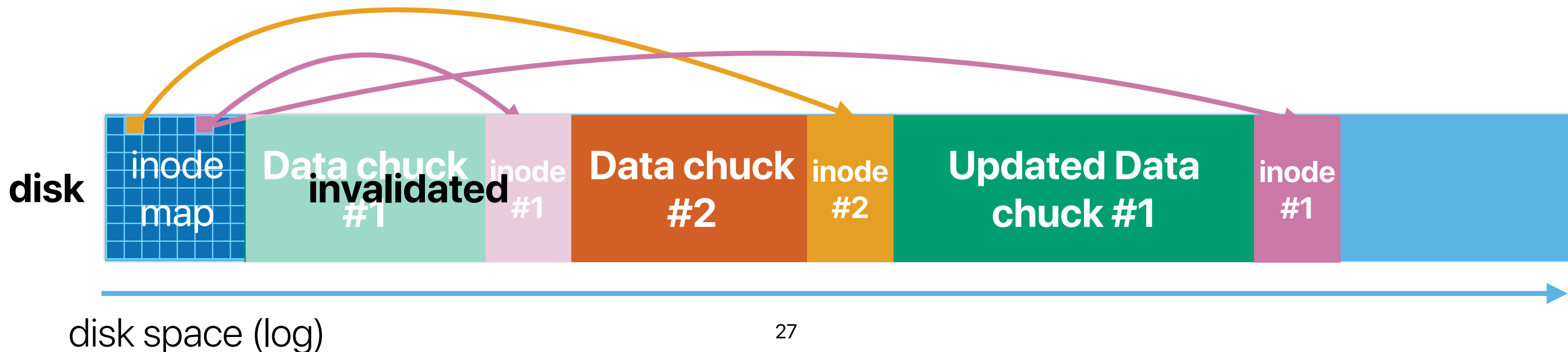


What's the good things and bad things about "Logs"

Pros/Cons of Logs

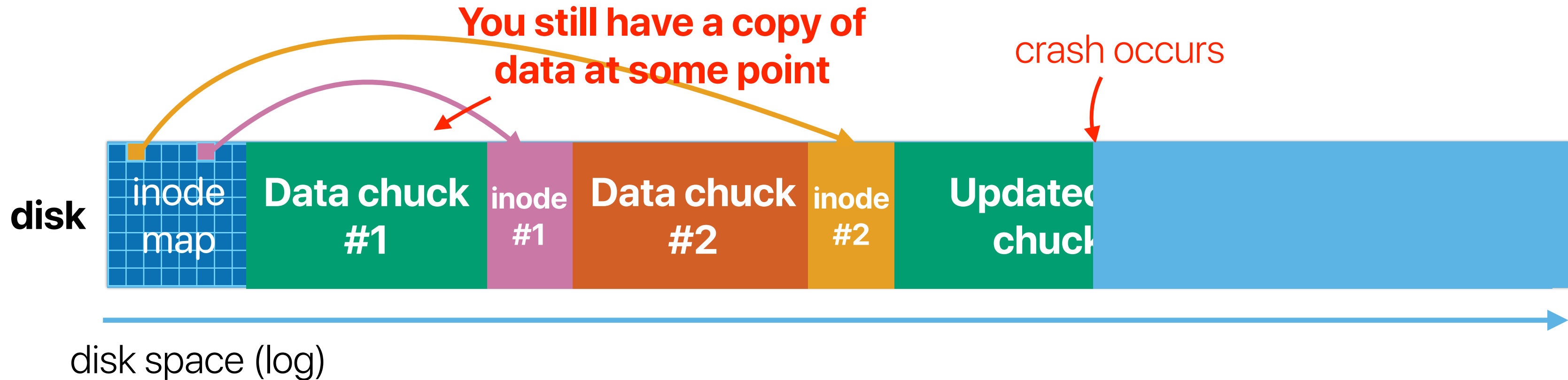
Log

- An append only data structure that records all changes
- Advantages of logs
 - Better performance — always sequential accesses
 - Faster writes — you just need to append without sanitize existing data first
 - Ease of recovery — you can find previous changes within the log
- Disadvantage of logs — you will need to explicit perform garbage collections to reclaim spaces

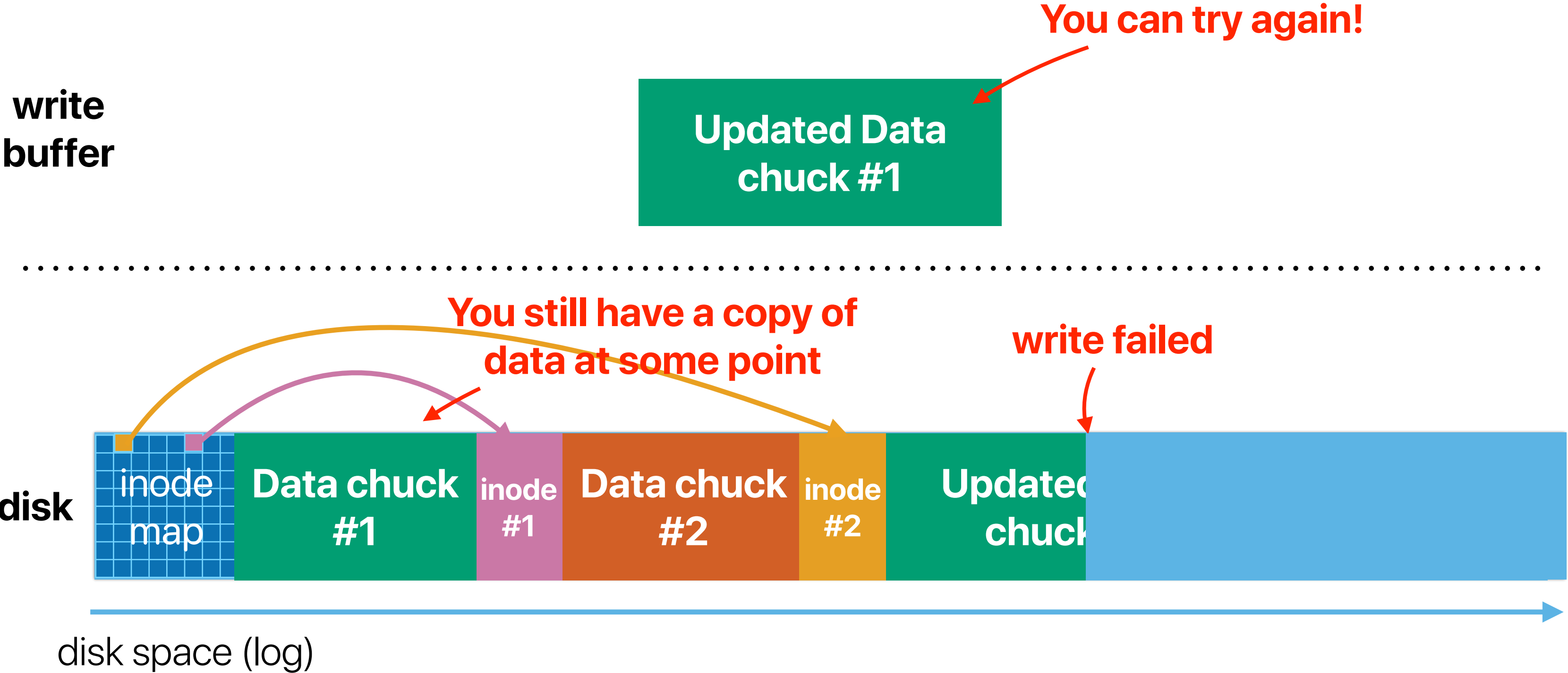


LFS v.s. crash

write
buffer

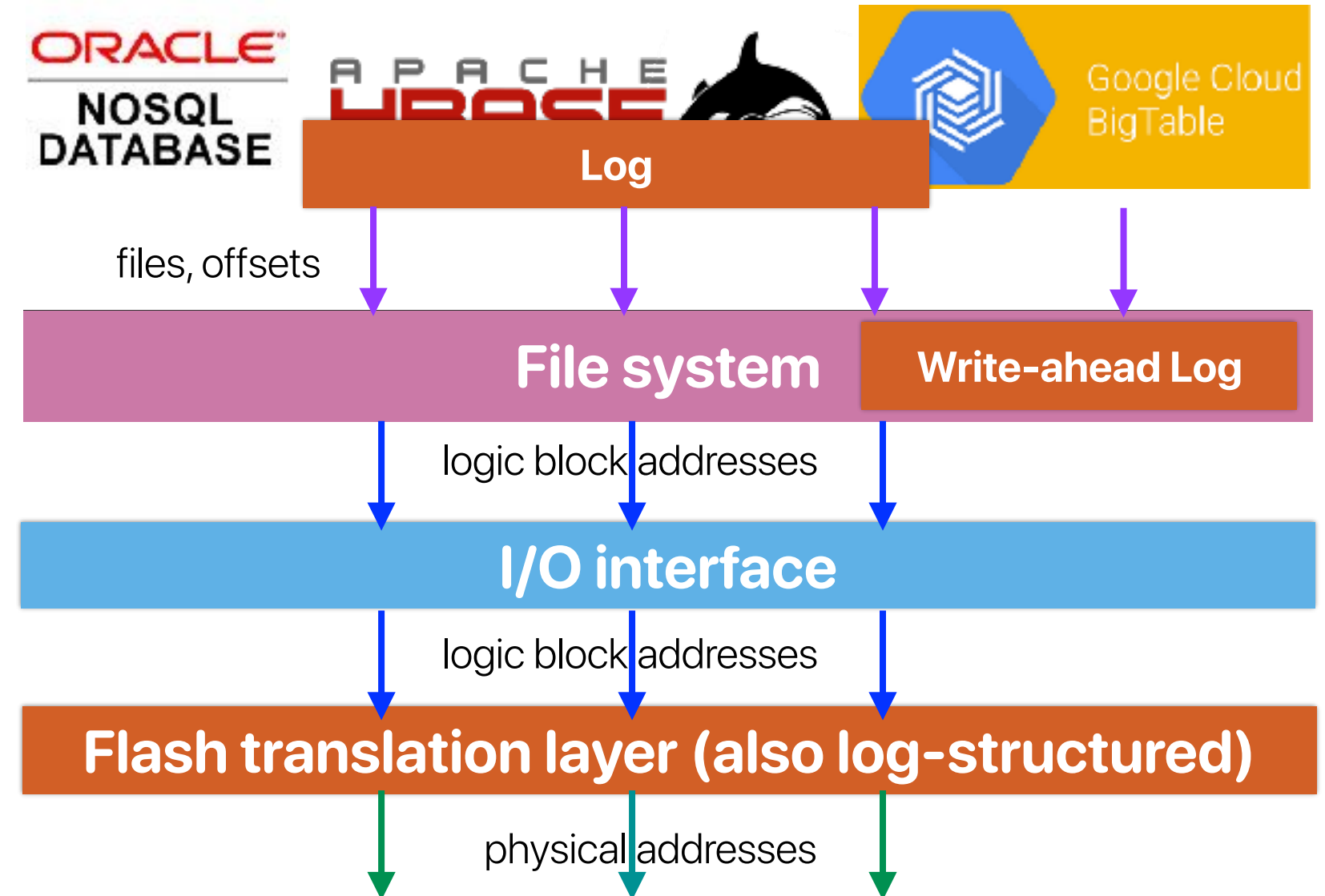


LFS v.s. write failed

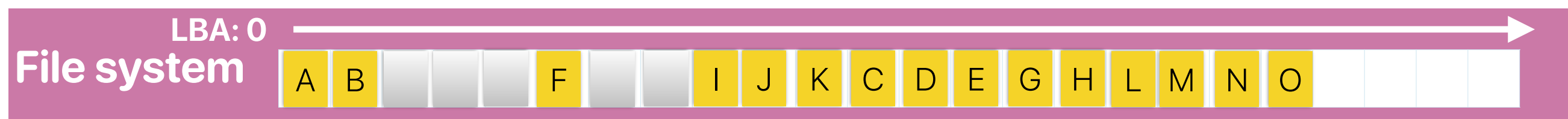


Why should we care about this paper?

- Log is everywhere
 - Application: database
 - File system
 - Flash-based SSDs
- They can interfere with each other!
- An issue with software engineering nowadays



For example, garbage collection

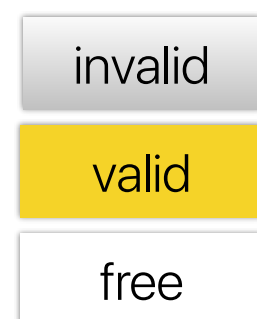
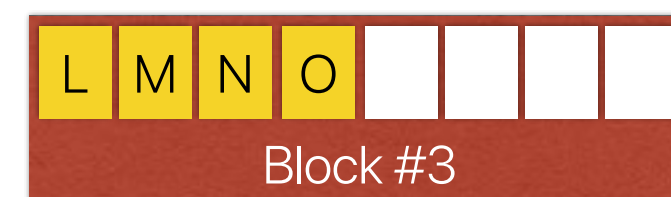
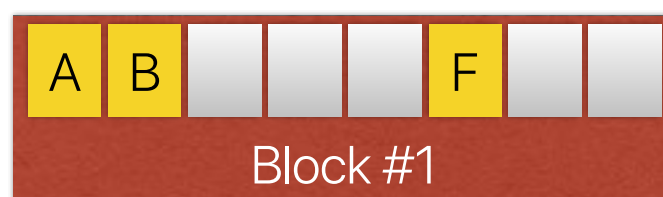


logic block addresses

I/O interface

logic block addresses

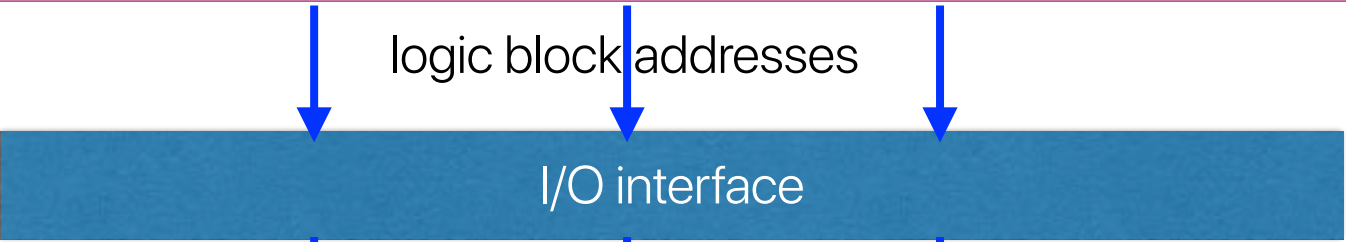
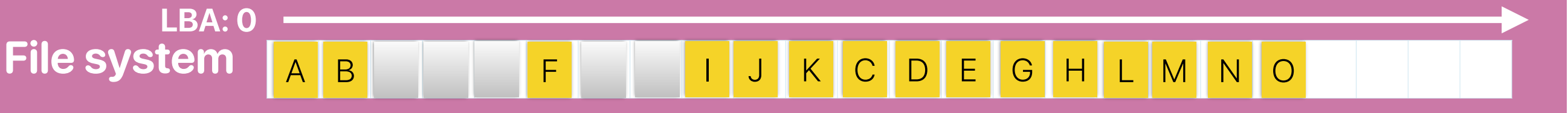
SSD



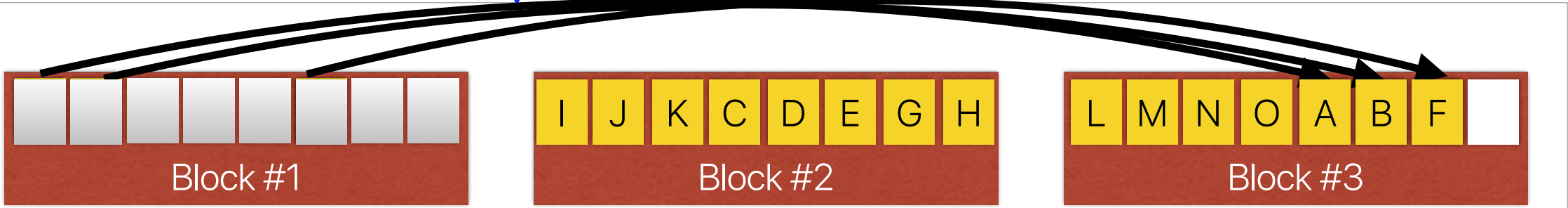
FTL mapping table

LBA	block #	page #
0	1	0
1	1	1
2	-	-
3	-	-
4	-	-
5	1	5
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	-	-
21	-	-
22	-	-
23	-	-

Now, SSD wants to reclaim a block



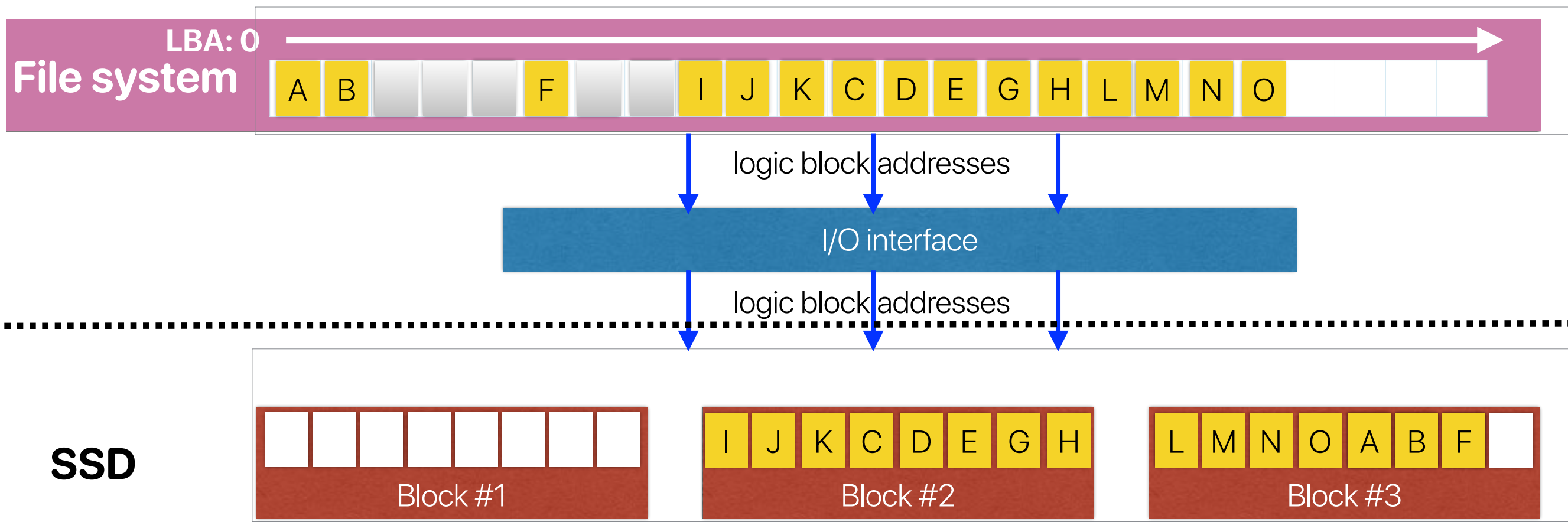
SSD



FTL mapping table

LBA	block #	page #
0	3	4
1	3	5
2	-	-
3	-	-
4	-	-
5	3	6
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	-	-
21	-	-
22	-	-
23	-	-

Garbage collection on the SSD done!



FTL mapping table

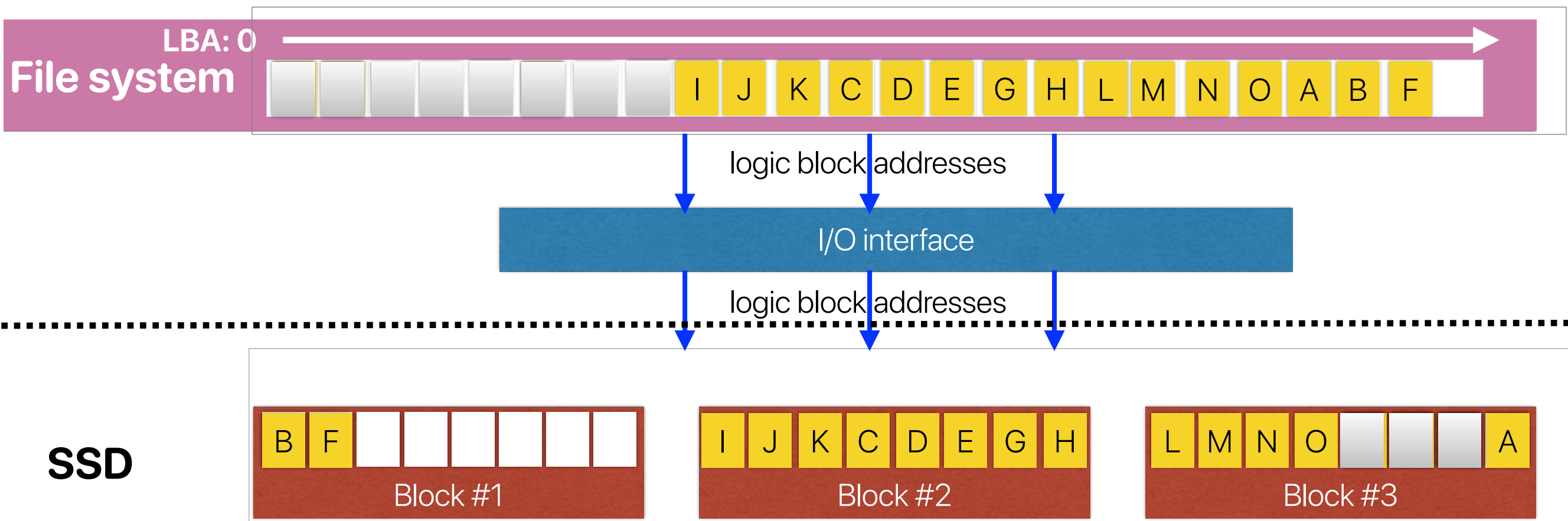
LBA	block #	page #
0	3	4
1	3	5
2	-	-
3	-	-
4	-	-
5	3	6
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	-	-
21	-	-
22	-	-
23	-	-

invalid

valid

free

What will happen if the FS wants to perform GC?



FTL mapping table

LBA	block #	page #
0	-	-
1	-	-
2	-	-
3	-	-
4	-	-
5	-	-
6	-	-
7	-	-
8	2	0
9	2	1
10	2	2
11	2	3
12	2	4
13	2	5
14	2	6
15	2	7
16	3	0
17	3	1
18	3	2
19	3	3
20	3	7
21	1	0
22	1	1
23	-	-

We could have avoided writing the stale A, B, F if they are coordinated!



All problems in computer science can be solved by another level of
indirection

—David Wheeler

...except for the problem of too many layers of indirection.

File systems for flash-based SSDs

- Still an open research question
- Software designer should be aware of the characteristics of underlying hardware components
- Revising the layered design to expose more SSD information to the file system or the other way around

BGR

TECH

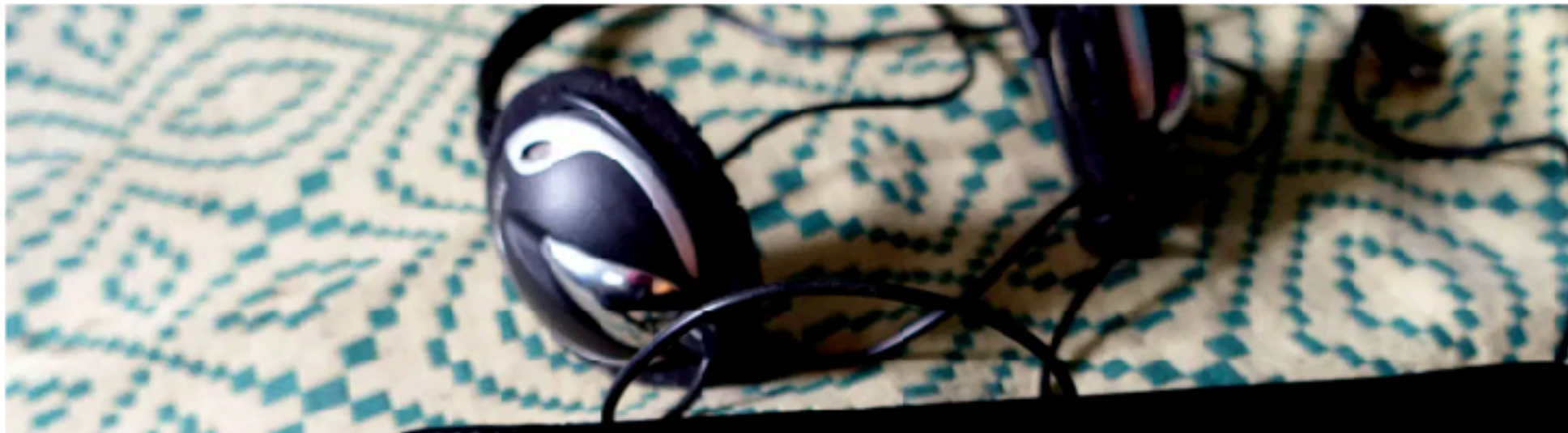
ENTERTAINMENT

DEALS

BUSINESS

TECH

Spotify has been quietly killing your SSD's life



Apple M1 Macs appear to be chewing through their SSDs

By Alan Dexter 4 hours ago

The same SSDs that are soldered-in and almost impossible to rep

COMMENTS



(Image credit: Apple)

A typical GPU code

storing input in m

```
1: m = (float*) malloc(sizeof(float)*matrix_dim*matrix_dim); // allocate m
2: fp = open(filename, O_RDONLY|O_DIRECT); // open file
3: read(fp, m, sizeof(float)*matrix_dim*matrix_dim); // read file
```

```
4: close(fp);
   // allocate GPU memory
```

copying m to GPU memory

```
5: cudaMalloc((void**)&d_m, matrix_dim*matrix_dim*sizeof(float));
   // copy from main memory to GPU memory
```

```
6: cudaMemcpy(d_m, m, matrix_dim*matrix_dim*sizeof(float), cudaMemcpyHostToDevice);
```

```
7: lud_diagonal<<<1, BLOCK_SIZE>>>(d_m, matrix_dim, i); GPU computing
```

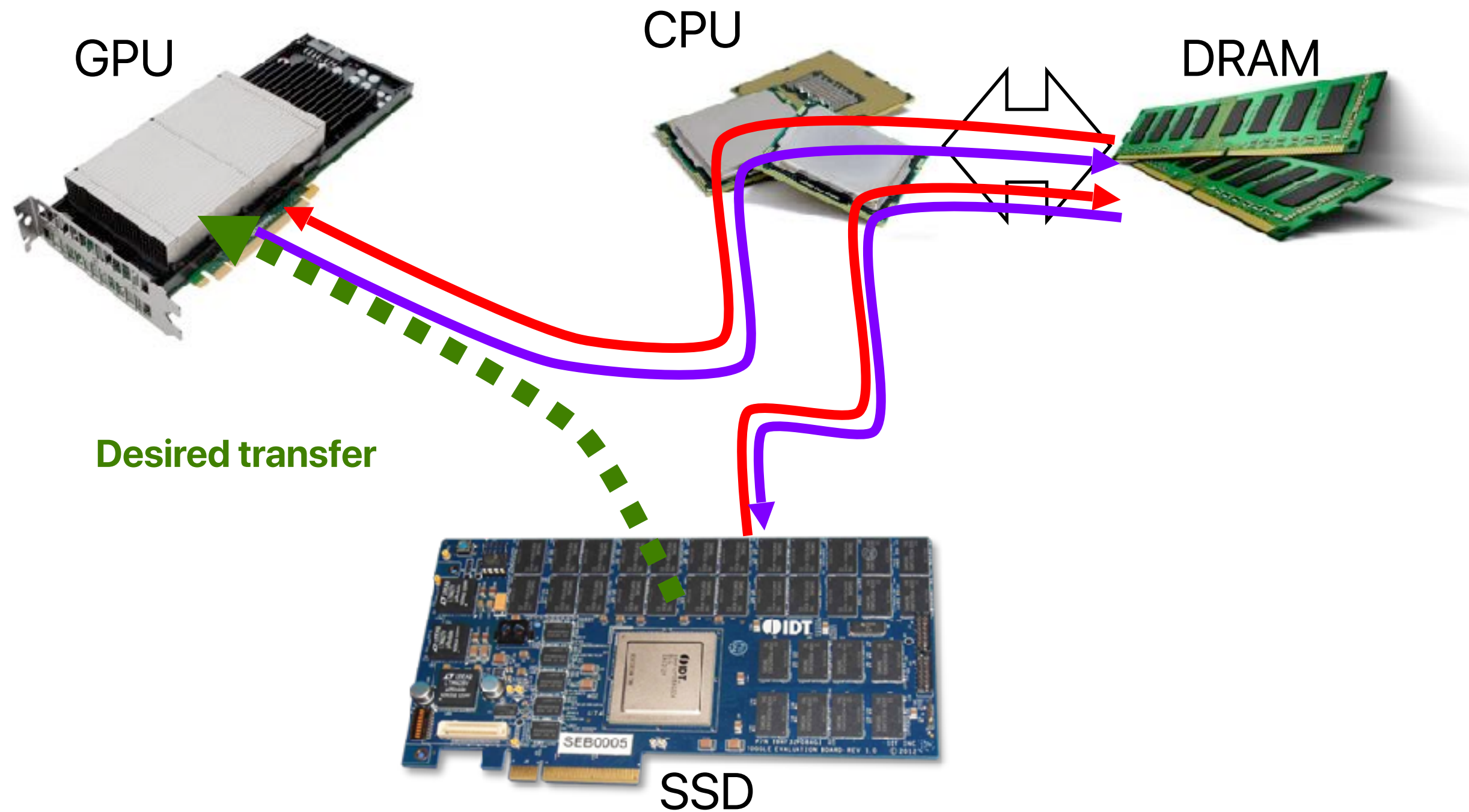
```
// copy from GPU memory to main memory
8: cudaMemcpy(m, d_m, matrix_dim*matrix_dim*sizeof(float),
   cudaMemcpyDeviceToHost);
9: output_matrix(m, matrix_dim);
```

Moving data

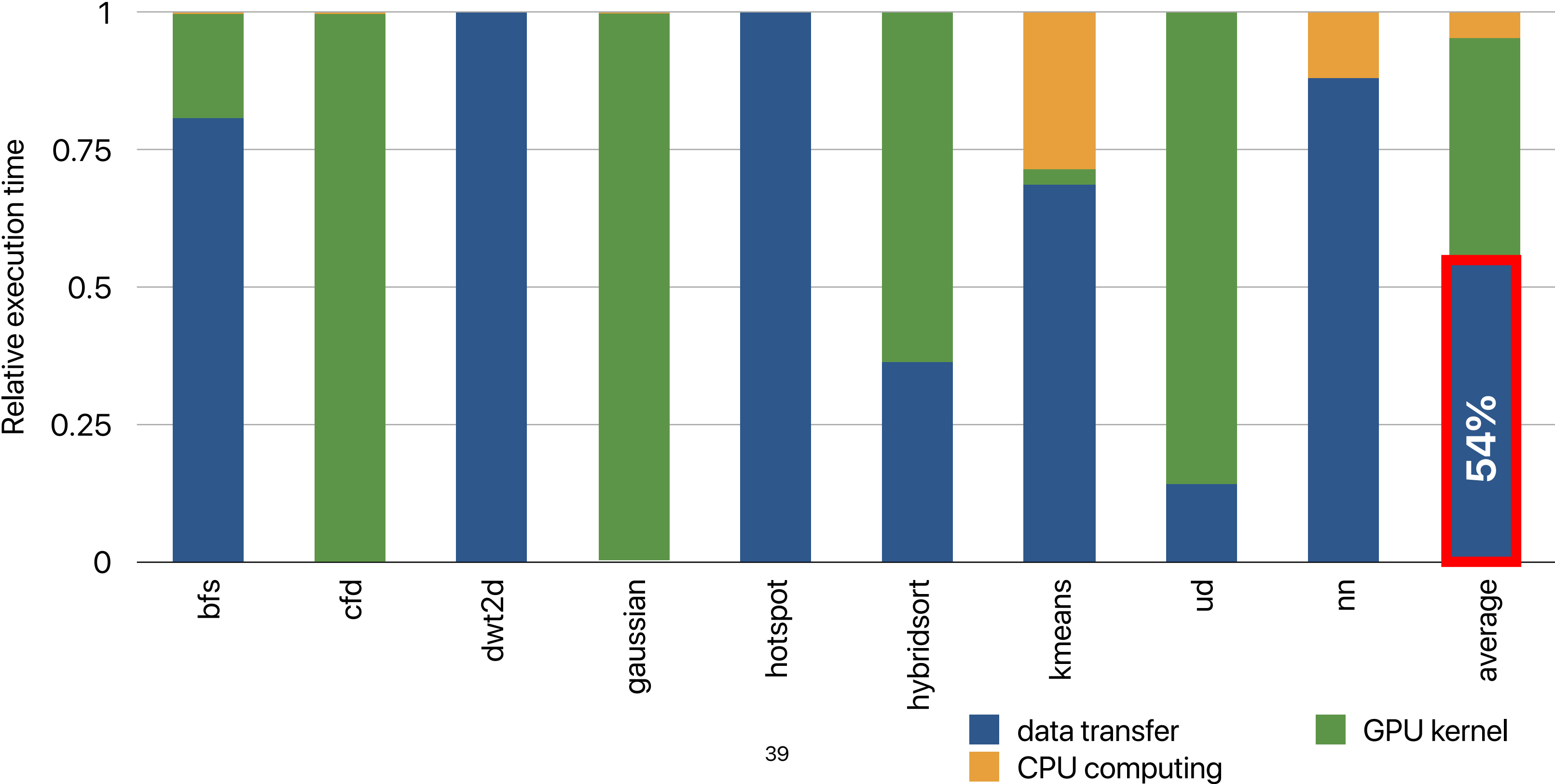
**m is overwritten, previous m is
only used as a "buffer"**

Moving data

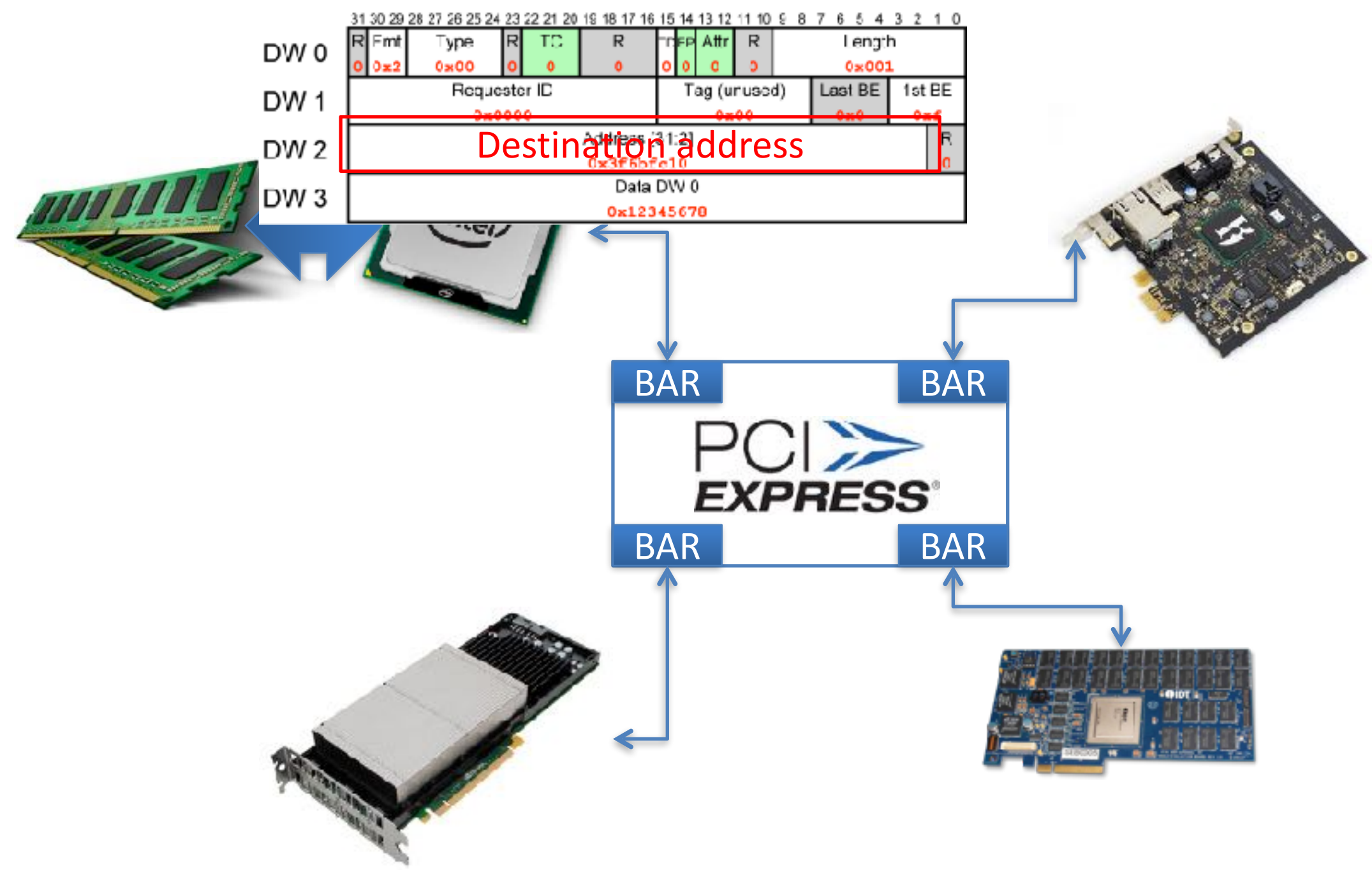
GPU-accelerated architecture



Cost of moving data



PCIe P2P

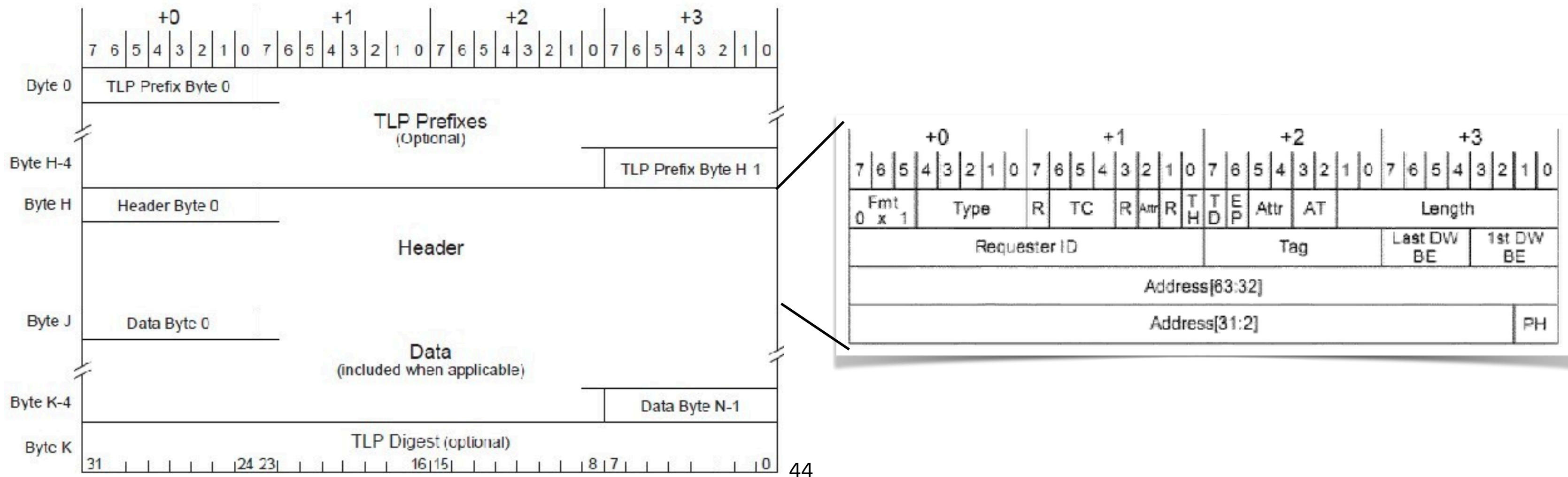


**How can you let SSDs to
communicate with GPUs directly &
why does SSD-GPU P2P make sense?**

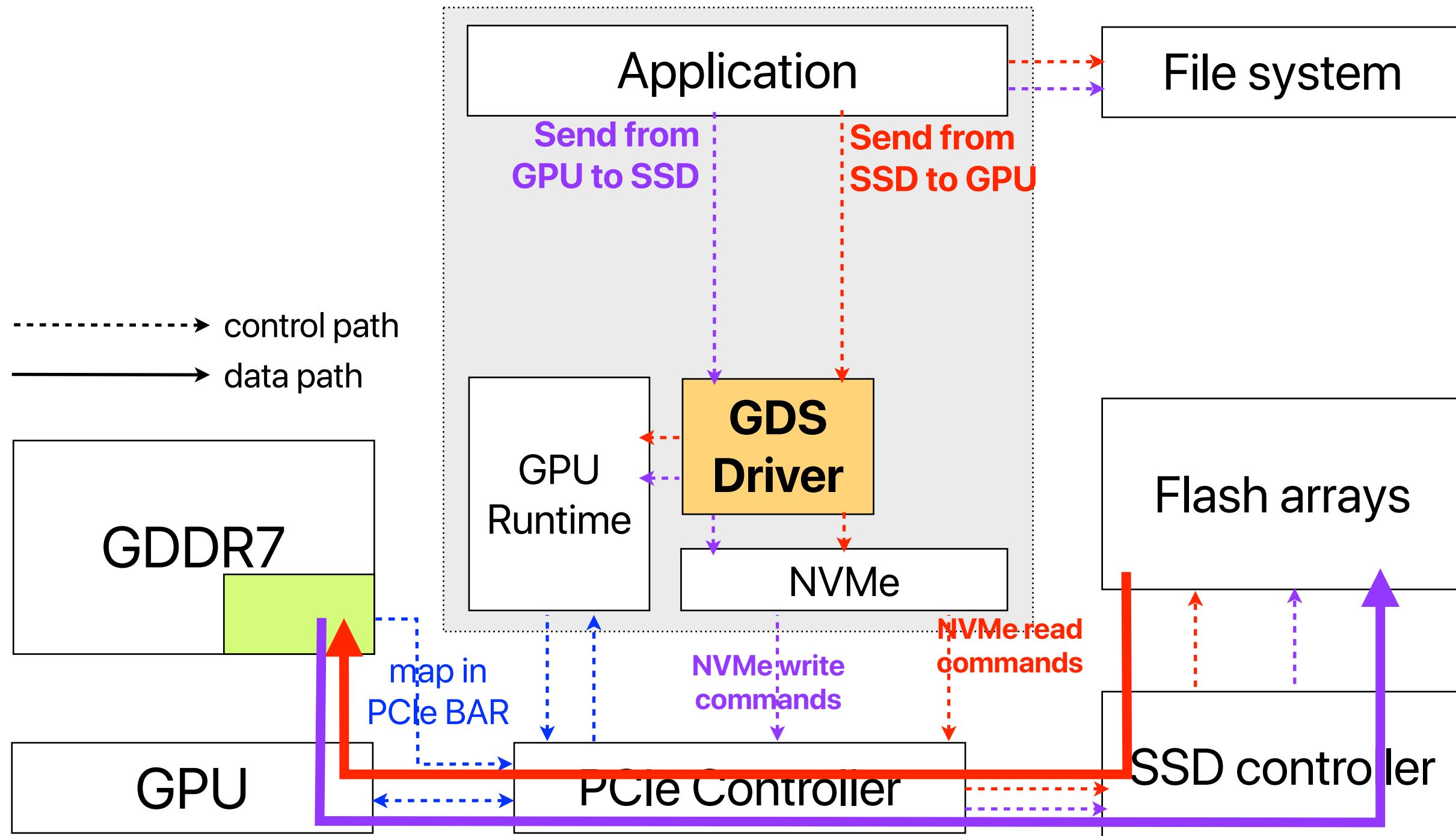
How to implement GPU-SSD P2P?

PCIe interconnect

- Very similar to computer networks
- Use "memory addresses" as the identifier for routing
- Peer-to-peer communication is possible



P2P between GPU and SSD

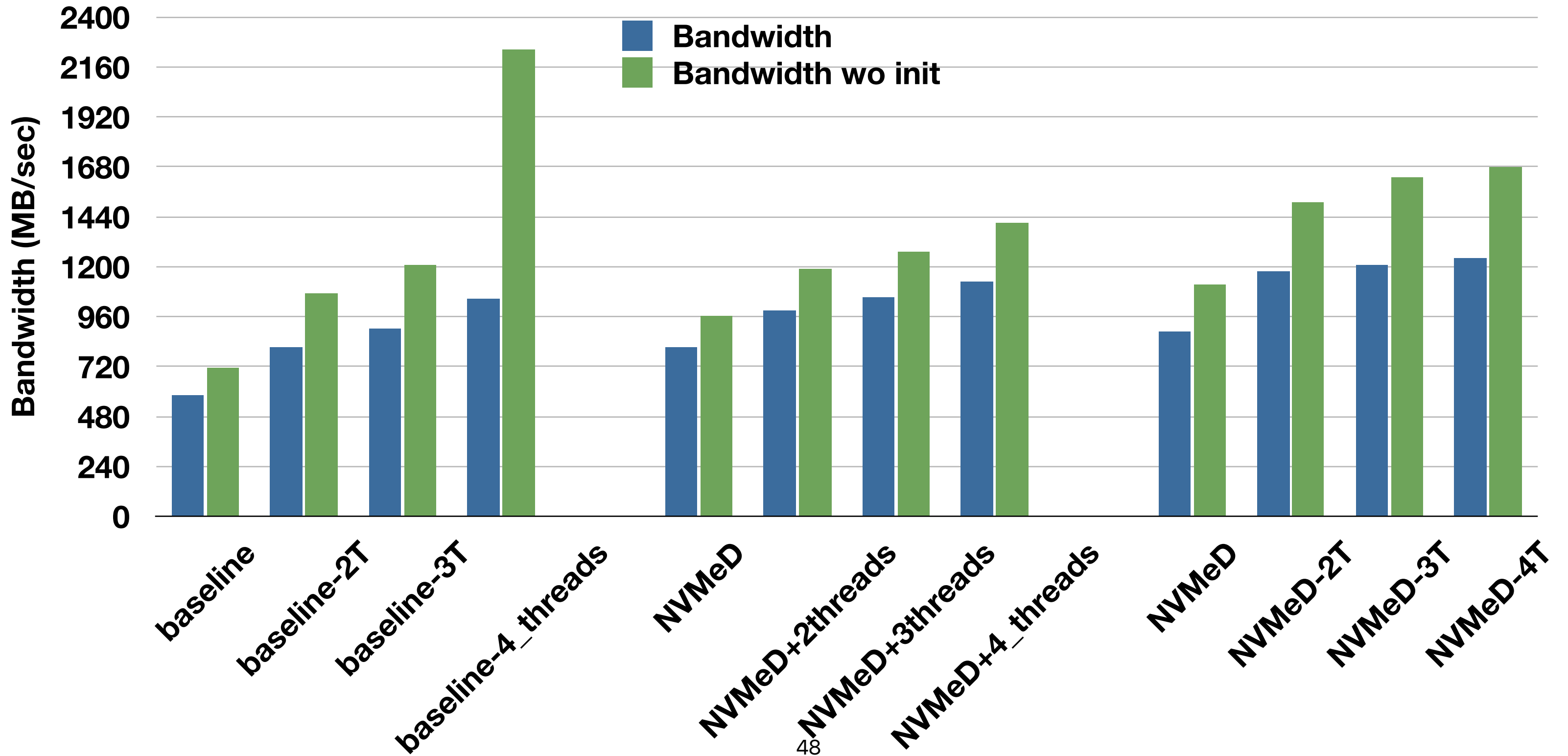


Code with GDS

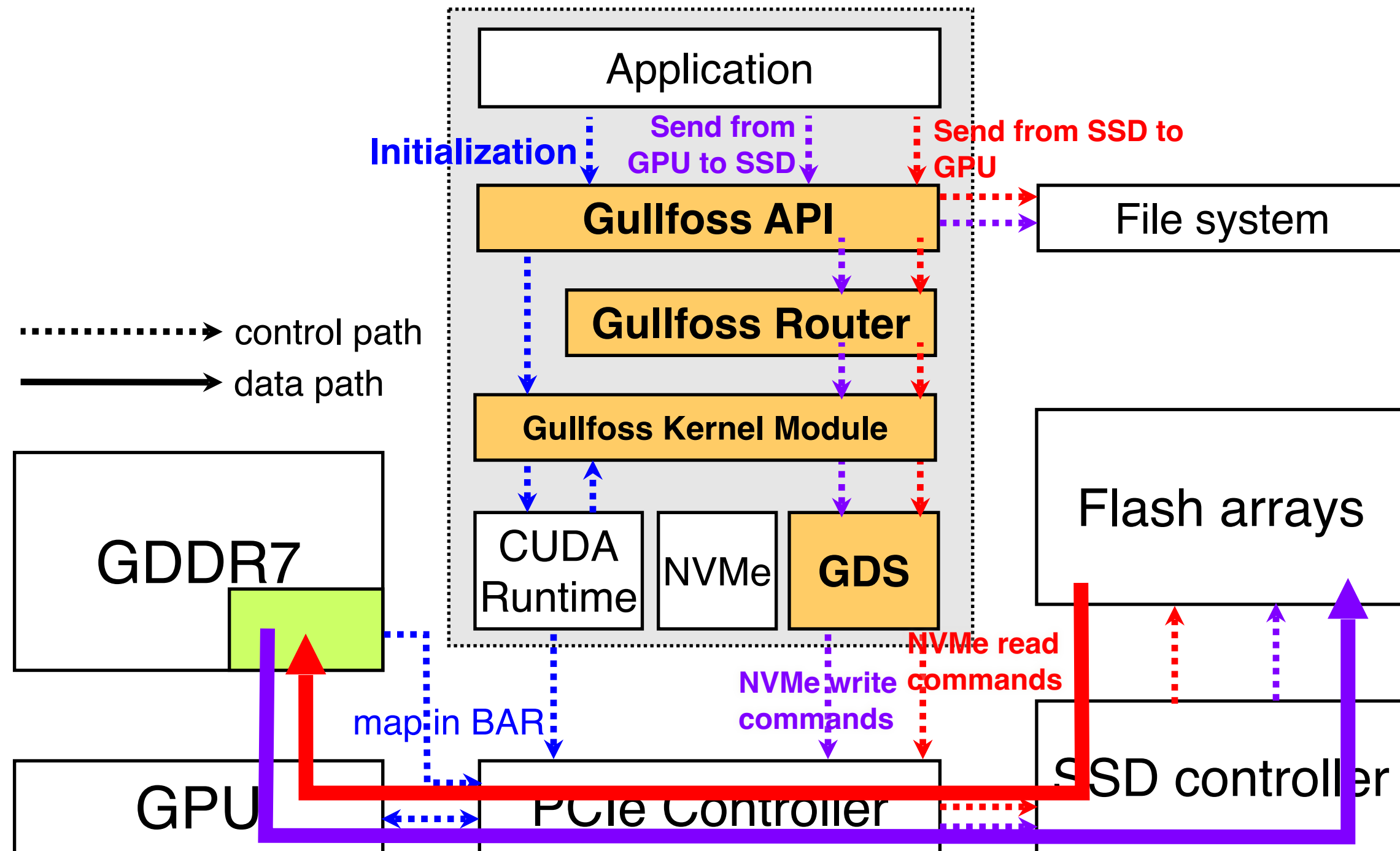
```
1: m = (float*) malloc(sizeof(float)*matrix_dim*matrix_dim); // allocate m
2: fp = open(filename, O_RDONLY|O_DIRECT); // open file
3: cfDescr.handle.fd = fd;
4: cfDescr.type = CU_FILE_HANDLE_TYPE_OPAQUE_FD;
5: CUfileError_t status = cuFileHandleRegister(&cfHandle, &cfDescr);
   // allocate GPU memory
6: cudaMalloc((void**)&d_m, matrix_dim*matrix_dim*sizeof(float));
7: ssize_t bytesRead = cuFileRead(cfHandle, d_m, matrix_dim*matrix_dim*sizeof(float), 0, 0);
   Moving data // read file directly to GPU
8: lud_diagonal<<<1,BLOCK_SIZE>>>(d_m, matrix_dim, i); GPU computing
   //copy from GPU memory to main memory
9: cudaMemcpy(m, d_m, matrix_dim*matrix_dim*sizeof(float), cudaMemcpyDeviceToHost);
10: output_matrix(m, matrix_dim); Moving data
```

storing output in m, no overwrite

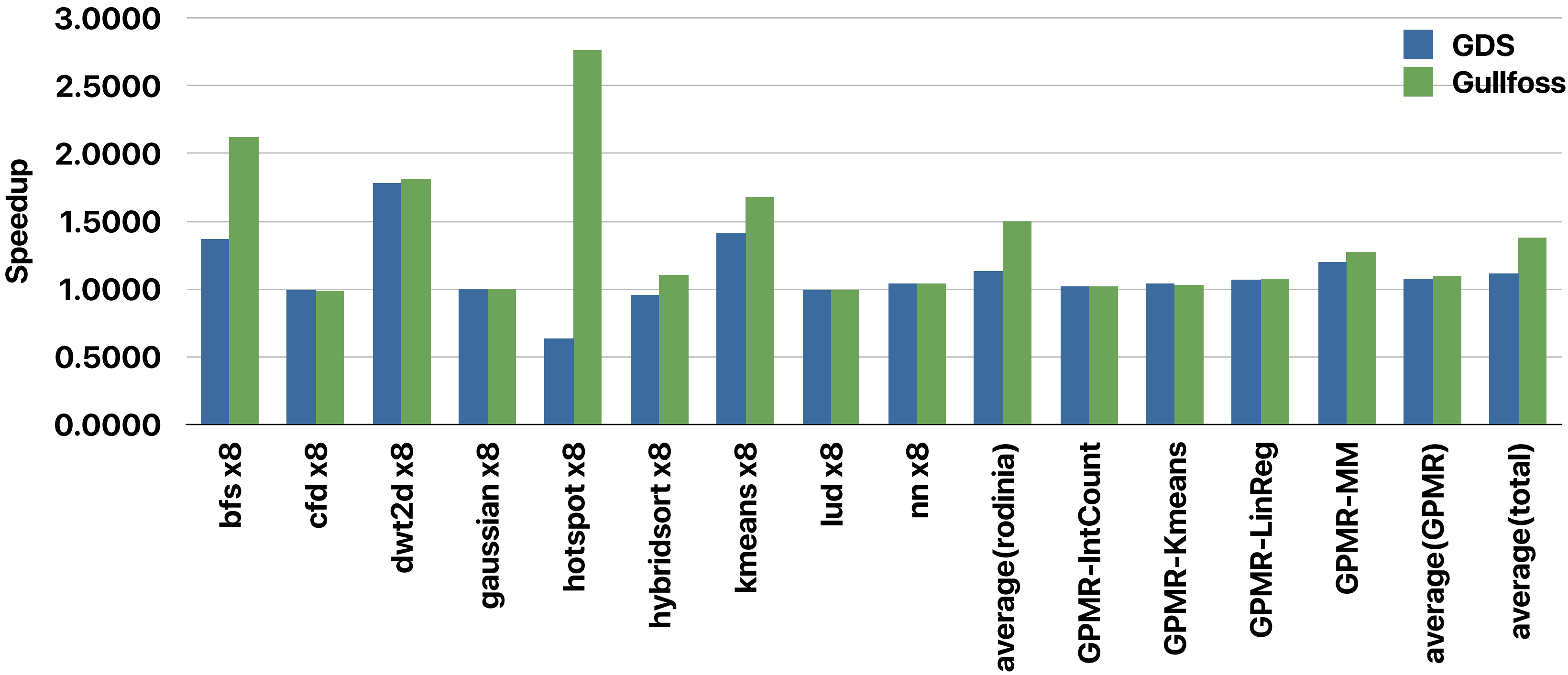
Multi-threaded I/O helps



Gulfoss: decouples control/data planes



GPU-SSD P2P + traditional route and main memory buffers



Google Final Project Presentations

- Make an appointment on **3/12** through the Google Calendar **ASAP**
- 12 minute presentation with 3 minute Q & A
- Why & what & how!!! — considering you're giving a presentation at Apple's keynote
 - 5-minute why & what — remind everyone what's your project about
 - 7-minute result — what lessons have you learned
 - Experimental result? Insights from the result?
 - Or why can't you get the experimental result?
 - Or why can't you get the result as you expected?
- Please reference this article to make a good presentation <https://cseweb.ucsd.edu/~swanson/GivingTalks.html>

Electrical Computer Science Engineering

277

つく

